Requirements Foundations

Part II

Context Impact

- The context has a **strong impact on the requirements** defined for a system
- **Inadequate** consideration of the context (e.g., ignoring, overlooking, or misconceiving aspects of the context) leads to **requirements defects**
- Without knowing the context, system requirements can neither be **correctly defined**, nor **correctly interpreted** and **understood**
- The requirements engineering context is thus a **cornerstone** of our requirements engineering framework

Context Impact

- The **context** determines which **solutions** are **appropriate** for realising the vision (and the requirements)
- **Context information** typically **restrict the solution space**
- The context influences
  - The **definition** of **requirements**
  - The **interpretation** of **requirements**

Example: Power Supply (1)

E **Vision**: Develop a power supply based on renewable energies.

Context 1: Family home in a countryside

→ **Wind turbine** is a **good solution**, provided there is enough wind in the area

All icons from [1]

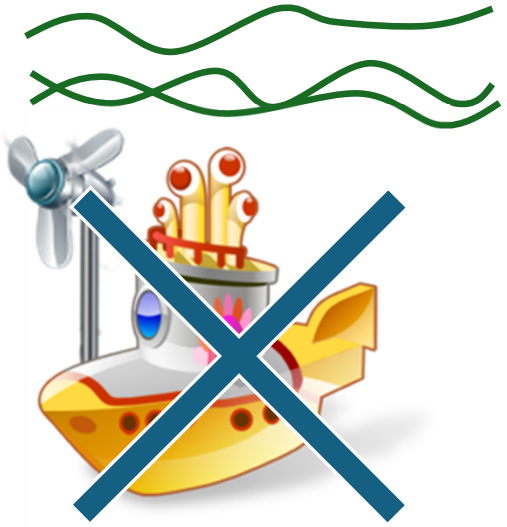
Context Impact – on Solutions

Example: Power Supply (2)

E **Vision**: Develop a power supply based on renewable energies.

Context 2: Submarine under water

→ **Wind turbine** is an **infeasible solution** (cannot be use under water)



All icons from [1]

Example: Power Supply (3)

E **Vision**: Develop a power supply based on renewable energies.

Context 3: Satellite in space

→ **Wind turbine** is an **infeasible solution** (there is no wind in space)

All icons from [1]

Context Impact

- The **context** determines which **solutions** are **appropriate** for realising the vision and requirements
- **Context information** typically **restrict the solution space**
- The context influences
  - The **definition** of **requirements**
  - The **interpretation** of **requirements**

Example: Transportation System (1)

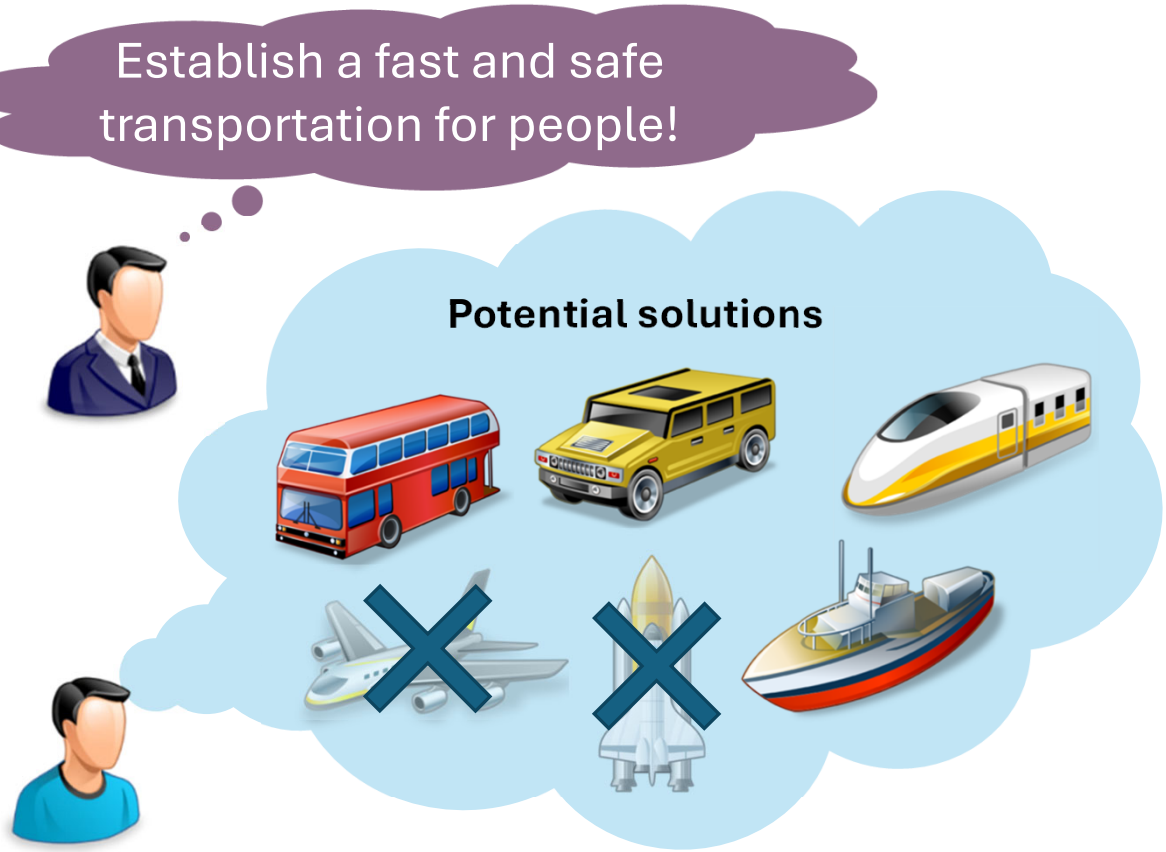
All icons from [1]

Example: Transportation System (2)

E

Context information:

- The transportation should be between the mainland and an island.
- The island is located 3 miles from the mainland.
- The island does not have an airfield.



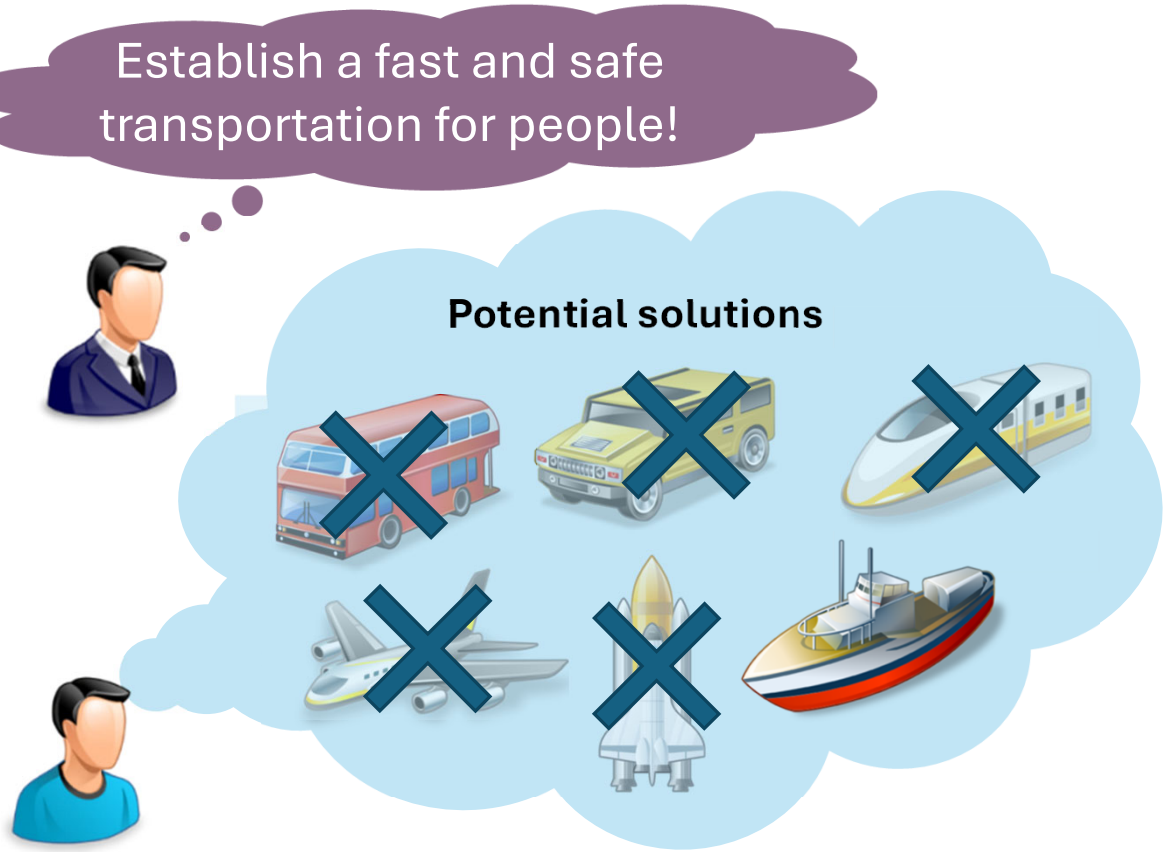
All icons from [1]

Example: Transportation System (3)

E

Context information:

- The transportation should be between the mainland and an island.
- The island is located 3 miles from the mainland.
- The island does not have an airfield.
- The island has about 35 inhabitants.
- The island is no major tourist destination.



All icons from [1]

Context Impact

- The **context** determines which **solutions** are **appropriate** for realising the vision and requirements
- **Context information** typically **restrict the solution space**
- The context influences
  - The **definition** of **requirements**
  - The **interpretation** of **requirements**

Context 1: Manufacturing Industry in Germany

E **Vision**: Develop an accounting system

In context 1 we have to consider:

- Relevant **German laws**
- Business needs of **manufacturing companies**

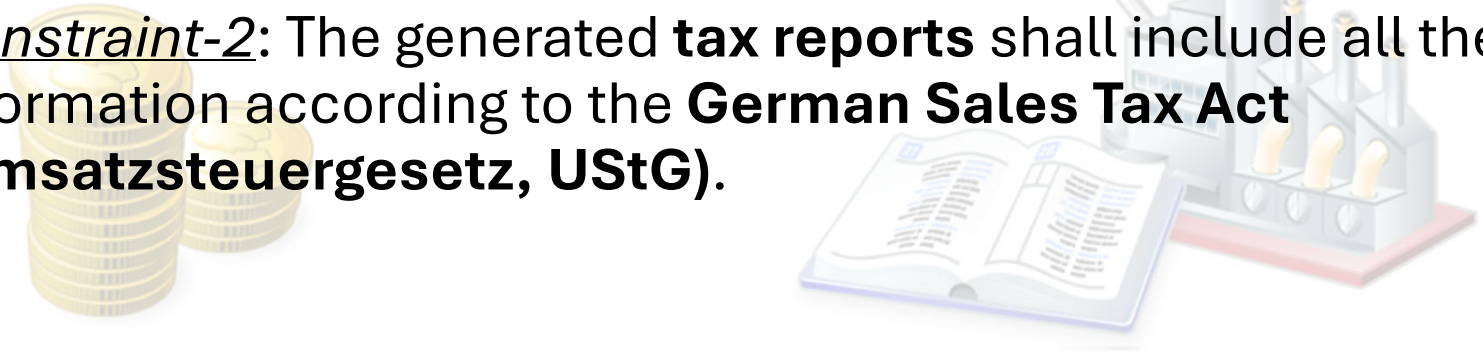
All icons from [1]

Context Impact – on Requirements Definition

Context 1: Valid Requirements for the German Manufacturing Industry

E

- *Functional-Req-121*: The accounting system shall calculate the **average production capacity** utilization over the last quarter to estimate future capacities.
- *Quality-Req-21*: The accounting system shall be able to interoperate with **common Enterprise Resource Planning (ERP)** systems used by **supplier companies**.
- *Constraint-2*: The generated **tax reports** shall include all the information according to the **German Sales Tax Act (Umsatzsteuergesetz, UStG)**.



Context 2: Financial Company in the USA

E **Vision**: Develop an accounting system.

In context 2 we have to consider:

- Relevant **US American laws**
- Business needs of the US **financial sector**

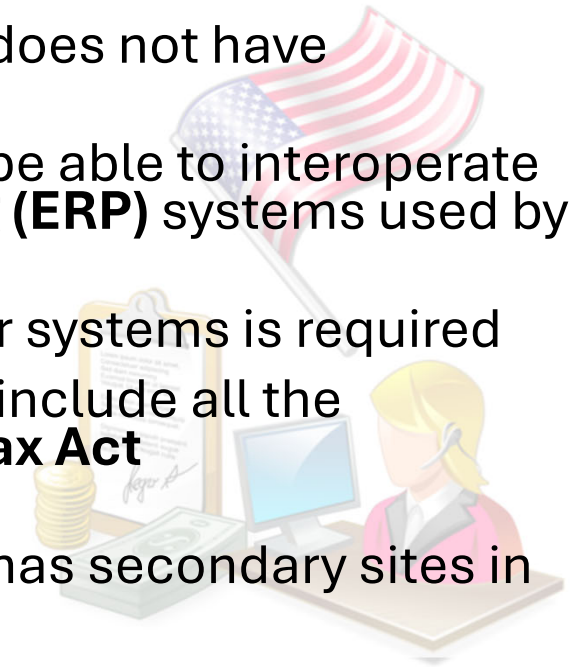
All icons from [1]

Context Impact – on Requirements Definition

Context 2: Are the Requirements defined for Context 1 Still Valid?

E

- *Functional-Req-121*: The accounting system shall calculate the **average production capacity** utilisation over the last quarter to estimate future capacities.
  - **Not valid**, because a financial company does not have production capacities
- *Quality-Req-21*: The accounting system shall be able to interoperate with **common Enterprise Resource Planning (ERP)** systems used by **supplier companies**.
- **Can be valid**, if interoperability with other systems is required
- *Constraint-2*: The generated **tax reports** shall include all the information according to the **German Sales Tax Act (Umsatzsteuergesetz, UStG)**.
- **Not valid**, unless the financial company has secondary sites in Germany

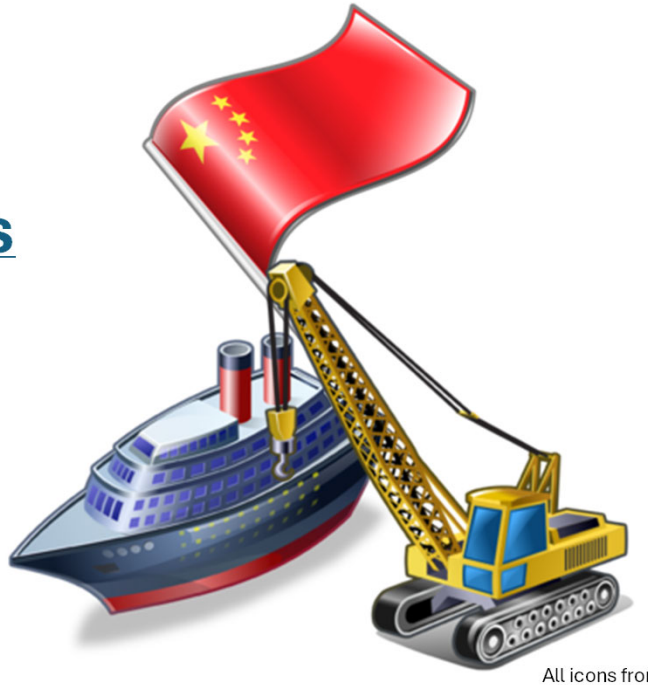


Context 3: Shipping Company in China

E **Vision**: Develop an accounting system.

In context 3 we have to consider:

- Relevant **Chinese laws**
- Business needs of Chinese **shipping companies**



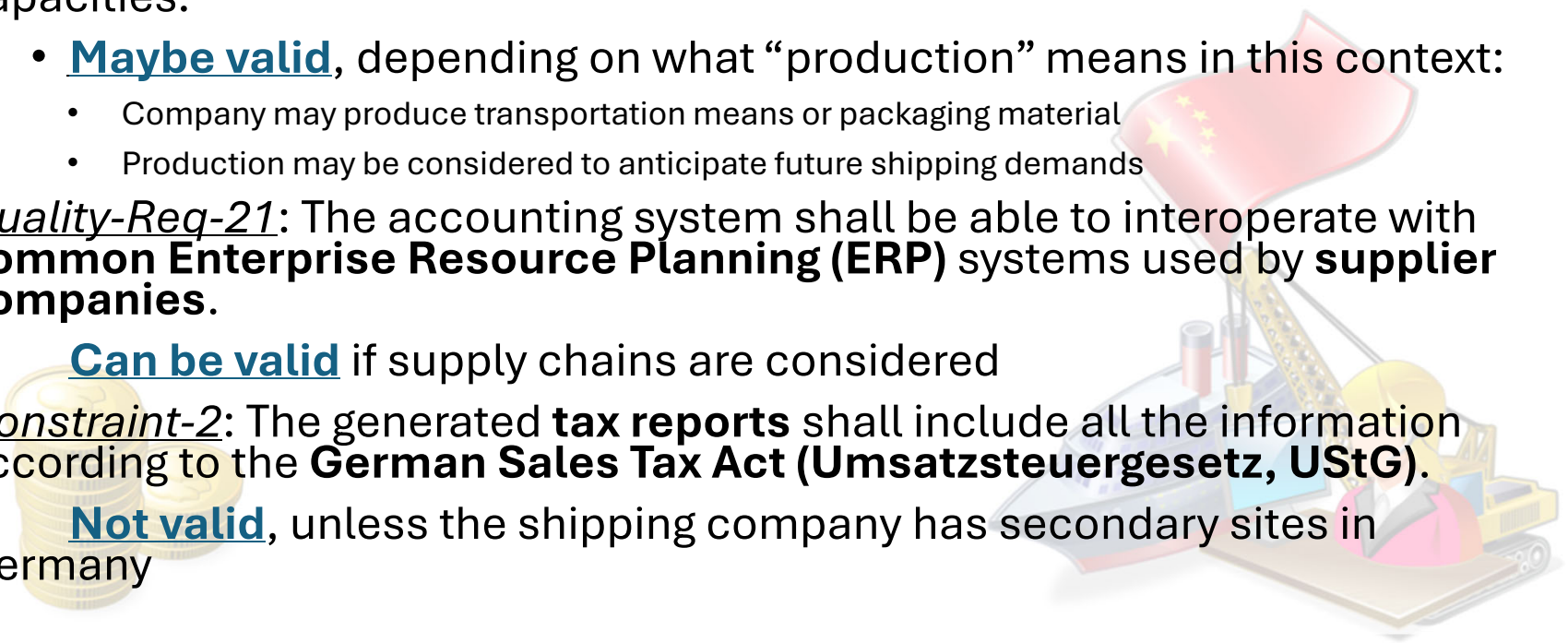
All icons from [1]

Context Impact – on Requirements Definition

Context 3: Are the Requirements defined for Context 1 Still Valid?

E

- *Functional-Req-121*: The accounting system shall calculate the **average production capacity** utilisation over the last quarter to estimate future capacities.
  - **Maybe valid**, depending on what "production" means in this context:
    - Company may produce transportation means or packaging material
    - Production may be considered to anticipate future shipping demands
- *Quality-Req-21*: The accounting system shall be able to interoperate with **common Enterprise Resource Planning (ERP)** systems used by **supplier companies**.
- **Can be valid** if supply chains are considered
- *Constraint-2*: The generated **tax reports** shall include all the information according to the **German Sales Tax Act (Umsatzsteuergesetz, UStG)**.
- **Not valid**, unless the shipping company has secondary sites in Germany



Context Impact

- The **context** determines which **solutions** are **appropriate** for realising the vision and requirements
- **Context information** typically **restrict the solution space**
- The context influences
  - The **definition** of **requirements**
  - The **interpretation** of **requirements**

Context Impact – on Requirements Interpretation

Example: User Authentication (1)

Functional-Req-42: The system **shall authenticate people** accessing the building or parts of it.

- A **software architect's interpretation**:
- The authorisation system **should control the access of people** entering the building
- **No need to identify each person** entering the building
  - Sufficient to use shared access codes for (groups of) people

All icons from [1]

Context Impact – on Requirements Interpretation

Example: User Authentication (2)

Functional-Req-42: The system **shall authenticate people** accessing the building or parts of it.

- The **project manager's interpretation**:
- The authorisation **system should control the access of people** entering the building (**same** as the software architect's interpretation)
- **Each individual person shall be identified** when entering the building (**different** from the software architect's interpretation)
  - Access to the reception at the ground floor need not to be controlled
  - After passing the reception area, individual control of people is required due to the high-security area on the 5th floor

All icons from [1]

Context Impact – on Requirements Interpretation

Example: User Authentication (3)

Functional-Req-42: The system **shall authenticate people** accessing the building or parts of it.

- A **union representative's interpretation**:
- The authorisation system **should not control the access** to the building (**different** from the software architect's and project manager's interpretations)
- **Guarantee a high degree of privacy for the people** (**different** from the software architect's and project manager's interpretations)
  - Access control should only be established for the where access control is really required
  - All other floors should not have any access control

All icons from [1]

Take Home Message

- The **context** determines which **solutions** are **appropriate** for realising the vision and the requirements
- **Context information** typically **restrict the solution space**
- The context influences
  - The **definition** of **requirements**
  - The **interpretation** of **requirements**

Definitions

Context Objects

D A context object is a **tangible** or **intangible object** relevant for **requirements engineering**. A context object **exists** in the **current** and/or in the **future reality**.

- Context objects
  - Have to be known and understood for **executing the requirements engineering activities**
  - Are **sources of information** (including requirements)
- **Omitting** a context object most likely leads to **missing** and/or **wrongly specified requirements**

Definitions

Context Information (1)

D Context information is the sub-set of **information** about the **reality** relevant for **requirements engineering**. Context information is typically **elicited** and – if required – **refined** during requirements engineering.

Examples of context information shown in earlier slides

Definitions

Context Information (2)

- Context information
  - Has to be considered in all **requirements engineering activities**
  - Is elicited from **requirements sources** (stakeholders, documents, existing systems, data)
  - Includes **context assumptions**:
    - About **future situations** in which the envisioned system is supposed to be **operating** (see, e.g., [IREB 2022; Zave and Jackson 1997])
    - About **user behaviour**, **expected workload** etc.
    - Can change over time – periodically need validation
  - Has to be **documented**

Definitions

Requirements Engineering Context

D The requirements engineering context subsumes all **context objects** and **context information** (relevant for requirements engineering).

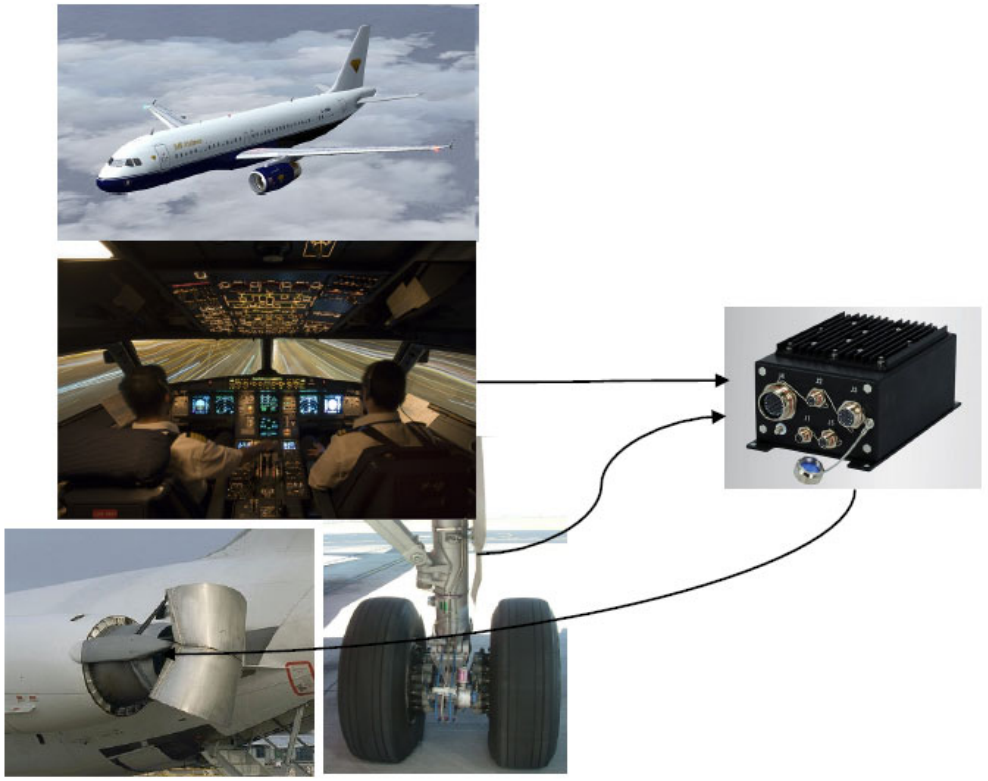
Context consideration is crucial **for requirements engineering** as well as **other development activities** (e.g., design and testing)

The World and the Machine

- The Machine denotes the product to be developed
  - E.g. the "software-to-be" + hardware on which it executes (Running a software transforms a general purpose computer into a specialized machine that performs specific tasks)
- The World denotes the part of the real world relevant to the development project
  - We build the machine to serve some practical purposes in the World

Phenomena

- A phenomenon is an observable state or event.
- Example
  - The plane is flying
  - Pilot pulls reverse thrust lever
  - Reverse thrust is on
  - …



World Phenomena vs. Machine Phenomena

- A world phenomenon is a phenomenon that can be observed in the World
- A machine phenomenon is a phenomenon that can be observed in the Machine
- A shared phenomenon is a phenomenon that can be observed at the interface between the World and the Machine

The World

The Machine

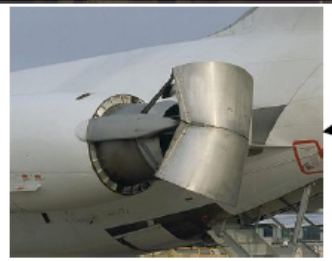
PlaneFlying

PlaneLanding

RevThrustOn

WheelsTurning

PilotActivatesReverseThrust



RevThrustCmd

WheelsSensorPulse

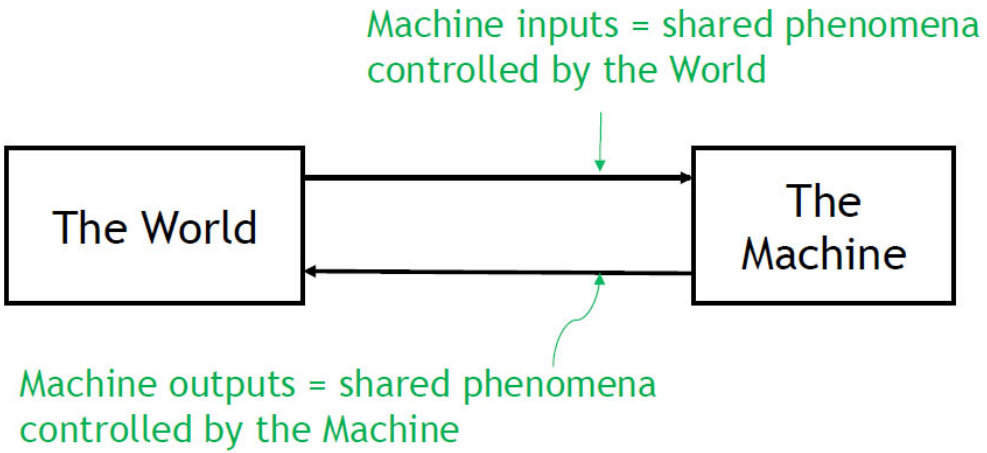
How Reverse Thrust Works:

https://www.youtube.com/watch?v=zcPVKVKPuRk

Control of Phenomena

- Shared Phenomena are either
  - Controlled by the World and **monitored** by the Machine

  Or
  - **Controlled** by the Machine



Example: Airplane Reverse Thrust Controller

- Reverse thrust should never be on while flying
- Reverse thrust should be on if the pilot activates it during landing

Non-Shared World Phenomena	Shared Phenomena	Non-Shared Machine Phenomena

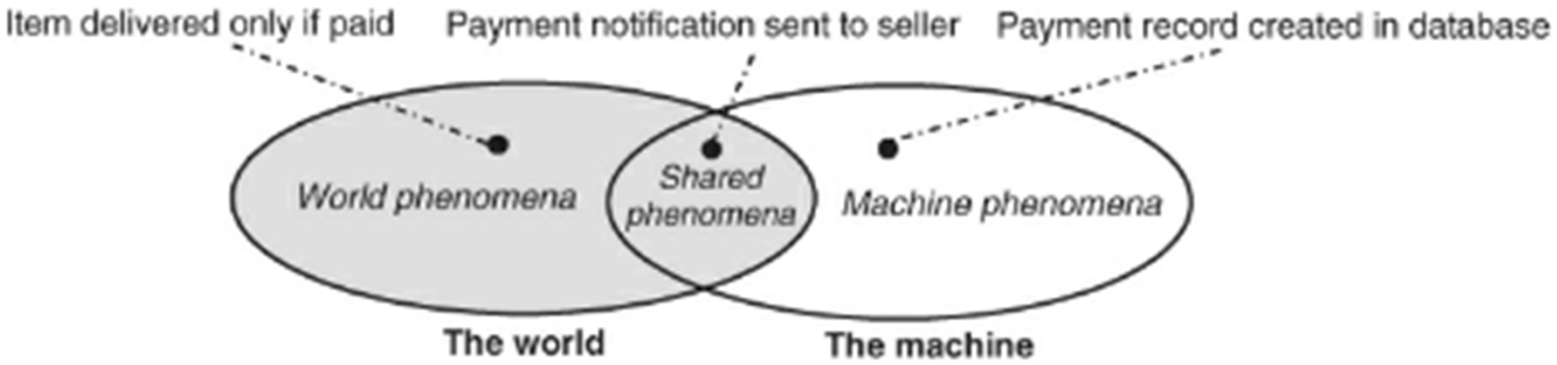
Example: Airplane Reverse Thrust Controller

- Reverse thrust should never be on while flying
- Reverse thrust should be on if the pilot activates it during landing

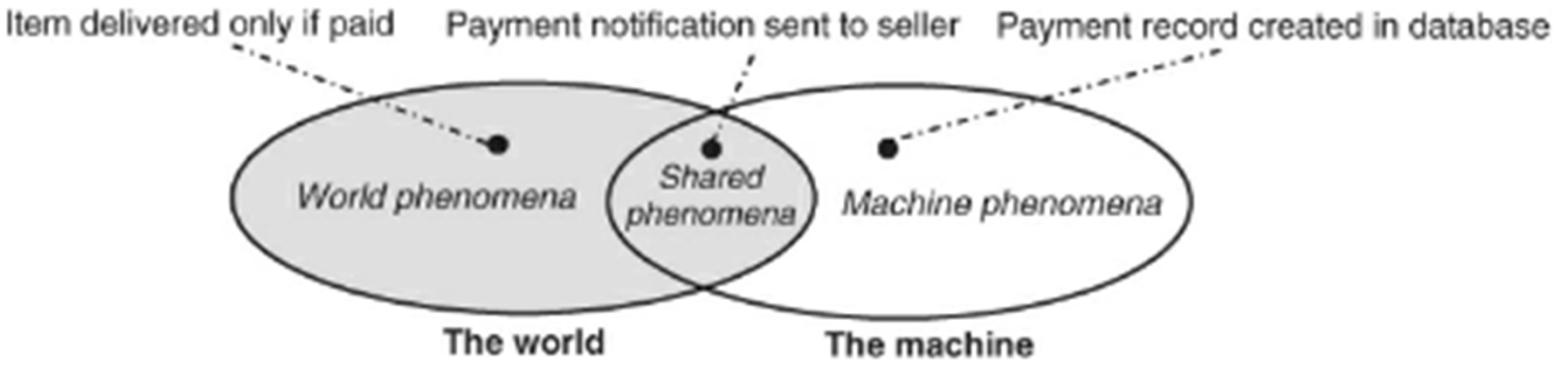
Non-Shared World Phenomena	Shared Phenomena	Non-Shared Machine Phenomena
ReverseThrustOn, PlaneFlying, PlaneLanding, WheelsTurning, MovingOnRunway	**Machine Inputs: (machine monitored)** PilotActivatesRevThrust WheelsSensorPulse	Program variable states, execution of code instructions,…
	Machine Outputs: (machine controlled) RevThrustCmd	

Another Example: An E-commerce System

- In this example, the World owns the phenomena of items being delivered to buyers only once they have been paid
- The Machine owns the phenomena of payment records being created in the machine's database.
- The phenomena of payment notifications being sent to sellers are shared, as the machine can control them whereas the world can monitor them.



Which parts of this diagram is RE mainly concerned with?



Quick reminder: Logical deduction

From A1, ..., An, I can logically deduce B1, ..., Bn

$\{A1, ..., An\} \vdash \{B1, ..., Bm\}$

The symbol $\vdash$ (called turnstile) can be read "therefore"

All men are mortal
Socrates is a man
$\vdash$
Socrates is mortal

If I press that button, the lights will turn off
If the lights turn off, the room will be dark
$\vdash$
If I press that button, the room will be dark

Fundamental Concepts

- Requirements
- Domain Assumptions
- Domain Properties
- Goals

Req, Dom |- Goal (the $E=mc^2$ of RE)

Fundamental Concepts

- Requirements
- Domain Assumptions
- Domain Properties
- Goals

Req, Dom |- Goal (the $E=mc^2$ of RE)

If the Machine satisfies its requirements **Req**
and
the World behaves as described in **Dom**,
then
all goals in **Goal** are satisfied.

Requirments Satisfaction Argument

`Req, Dom |- Goal`

If the Machine satisfies its requirements **Req**
and
the World behaves as described in **Dom**,
then
all goals in **Goal** are satisfied.

Goal

- A goal is a statement
  - Formulated entirely in terms of World phenomena
  - To be enforced by Machine in collaboration with other agents in the World
  - Synonyms: system requirement, system goal, business goal
- Example
  - The first ambulance must arrive at the incident scene within 14 minutes after the first call reporting the incident in at least 95% of cases.

Software Requirement

- A software requirement is a statement
  - Formulated entirely in terms of shared phenomena
  - To be enforced by the Machine alone
  - Synonyms: software specification
- Example
  - When the dispatcher confirms the allocation of an ambulance to an incident, the Computer Aided Dispatch software (CAD) shall send the mobilization instructions to the ambulance's Mobile Data Terminal (MDT) within 1 second.

Domain Assumptions

- A domain assumption is a statement
  - Formulated entirely in terms of World phenomena
  - To be enforced by World components other than the Machine
- Example
  - Ambulance drivers must signal arrival at the incident scene on the ambulance Mobile Data Terminal

Key Insight

- The distinction between requirements and implementation decisions is not a question of What vs. How
- The distinction between requirements and implementation decisions is a question of Where:
  - A requirement is a statement that talks about World phenomena
  - An implementation decision is a statement that talks about Machine phenomena

Descriptive vs. Prescriptive Statements

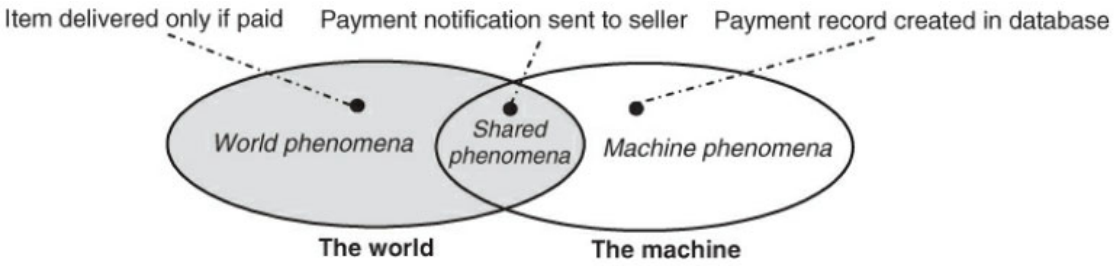
- Descriptive Statements indicate facts or beliefs
  - London has a population of 8 million habitants.
  - The London Ambulance Service receives an average of 3,800 calls per day
- Prescriptive Statements indicate wishes
  - In at least 95% of cases, an ambulance *should* arrive at the incident scene within 14 minutes after the first call reporting the incident.

Types of statements in RE

- System requirement
  - A prescriptive statement about a system
  - Formulated entirely in terms of World phenomena
  - To be enforced by Machine possibly in cooperation with other system components
  - Synonyms: **goal**
- Domain property is a
  - Descriptive statement about a system
  - Formulated entirely in terms of World phenomena

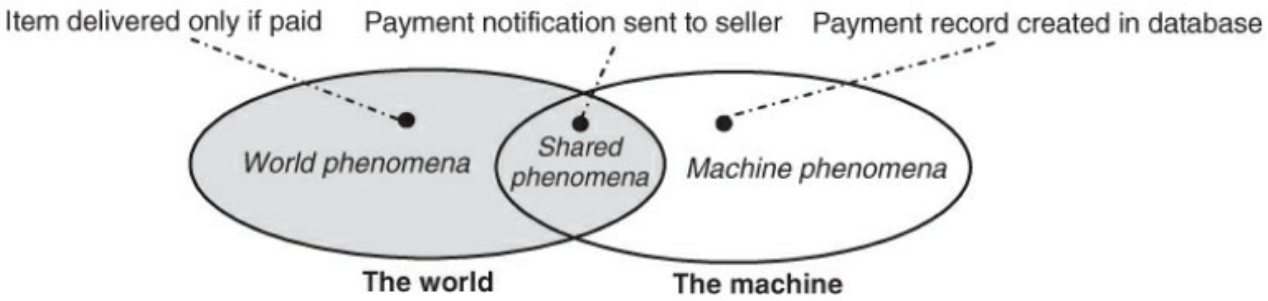
Types of statements in RE

- A software requirement is
  - A prescriptive statement about a system
  - To be enforced by the machine alone
  - Formulated entirely in terms of shared phenomena such that
    - It constraints only Machine-controlled phenomena and
    - It does refer to future values of monitored phenomena
  - Synonyms: machine specification, software specification



Types of statements in RE

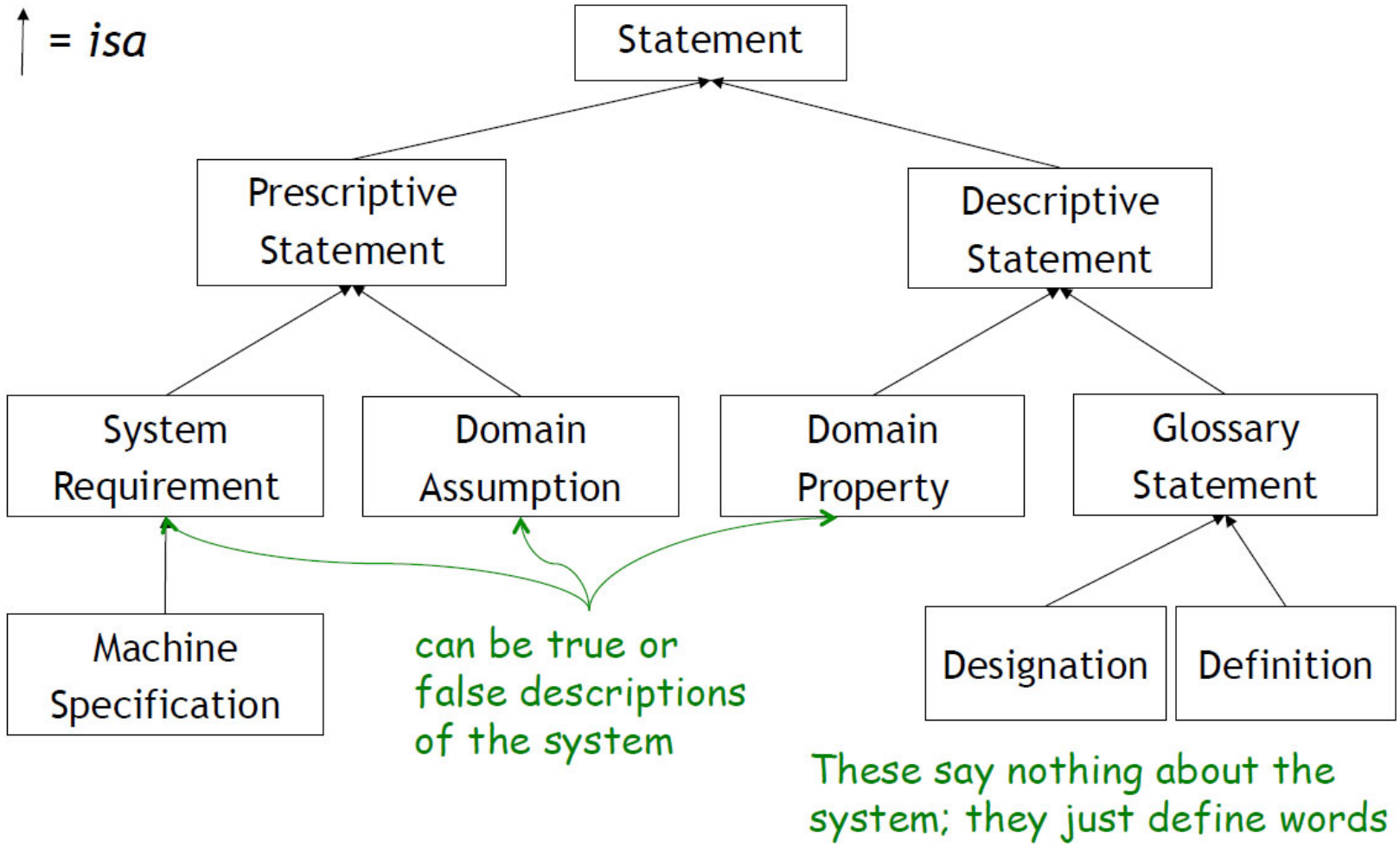
- A domain assumption is a
  - Prescriptive statement about a system
  - Formulated entirely in terms of World phenomena
  - To be enforced by system components other than the Machine



Types of statements in RE

- A designation is an information description of what a term represents in the World
  - The reporting time of an incident is the time at which the incident is first reported to the ambulance service
  - The intervention time of an incident is the time at which the first ambulance arrives at the incident scene
- A definition is a formal description that defines the meaning of a term in terms of other defined or designated terms.
  - The response time of an incident is defined as the amount of time between its intervention time and its reporting time:
    - Response time = intervention time – reporting time.

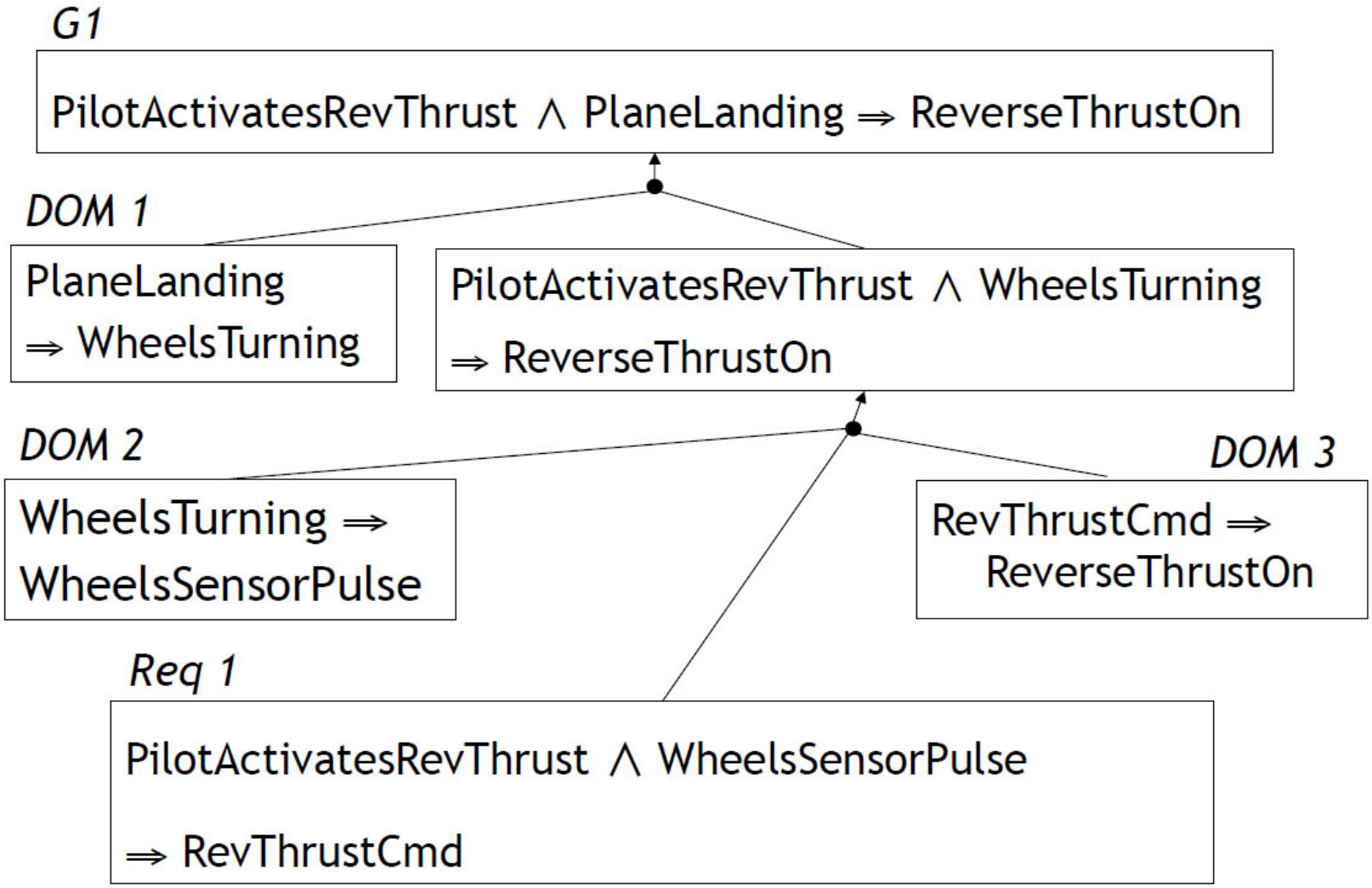
Classification of statements involved in RE



Reverse Thrust Example

- Goal
  - Reverse thrust should be on if the pilot activates it during landing
- Domain Assumption
  - When the plan is landing, its wheels are turning
  - When the wheels are turning, the wheels sensor pulse is ‘On’
  - When the RevThrustCmd is issued, the reverse thrust is turned on.
- Machine Requirements
  - If the pilot activates reverse thrust and the wheels sensor pulse is on, then the Reverse Thrust Controller shall send the RevThurstCmd signal

Example: Satisfaction Argument for Rev. Thrust Controller



Impact of Incorrect Domain Assumptions

- Many requirements errors are caused by invalid domain assumptions.

Example of Invalid Domain Assumption

DOM 1

PlaneLanding ⇒ WheelsTurning

If the plane is landing then its wheels are turning.

- Warsaw, 14 September 1993, property DOM1 did not hold. Plane skid on wet runway. This inhibited reverse thrust for 9 seconds. (Exact causes of accident more complex than this simplified example)

Example: London Ambulance Service

- System Requirement
  - [REQ1] An ambulance must arrive at incident scene within 14 minutes after first call.
- Domain Properties
  - [DOM1] Call takers encode incident's details and location.
  - [DOM2] GPS system gives ambulances' locations.
  - [DOM3] When an ambulance is allocated to an incident, ambulance crew will drive ambulance to incident location.
  - [DOM4] When ambulance arrives at location, ambulance crew signal arrival on Mobile Data Terminal.
  - …
- Machine Specification
  - …

Example of Invalid Domain Assumption

- Inaccurate or missing ambulance locations
- Crews taking different ambulance than allocated
- Crews pushing buttons on MDT in wrong sequence

- Oct 1992: severe system failures with long delays in ambulance responses (up to 11 hours!); software no longer knowing which ambulance was available or not, or where ambulances are; need to revert to manual system. But the software itself did not fail, it satisfied its requirements.

Good Requirements include

- Goals, domain properties, machine specs, domain assumptions.
- Include definitions and designations for all terms
- Check that requirements correspond to real needs
- Check that domain properties are valid
- Show that Dom, Spec |- Goal
- Don’t confuse system requirements with project requirements
  - The first is about system behaviors, the latter is about the development process.
- Don’t confuse system requirements with software models
  - The first is about World phenomena, the latter is about what goes on in the Machine

Role of Theory in RE

- It clarifies different types of statements involved in RE and the importance of distinguishing them.
- Clarifies the role and importance of domain assumptions in development; many errors are caused by incorrect domain assumptions.
- It defines the content and structure of an ideal requirements document.

An ideal requirements document

- A complete requirements description should contain
  - A set **Goal** describing the system requirements
  - A set **Spec** describing the Machine specifications
  - A set **Dom** describing the set of valid domain properties and assumptions
  - A glossary with **definition** and **designation** for all terms used in Goal, Spec, and Dom

- These statements must satisfy the satisfaction argument

Spec, Dom |- Goal

Furthermore, (Spec, Dom $\nvdash$ False)